

Without providing code, what drafter methods can be used to add content to a page, python

In the **Drafter** educational web development library for Python, pages are represented by the `Page` dataclass/object. Rather than using specialized Drafter-specific mutation methods on a page, Drafter structures page content as a Python list of elements (strings and UI components). GitHub Pages + 1

The primary ways to add content to a page include:

1. `Page` Constructor / Instantiation

Direct Content Initialization: Passing the content elements directly as a list argument when instantiating `Page` (e.g., providing the application state followed by a list of components or strings, or just the list for stateless routes). arXiv

2. Standard Python List Methods on the Content List

Because a `Page` 's content is held in a standard Python list, all built-in mutable sequence methods can be used to add items before returning the `Page` :

`append()` : Adds a single item (such as a string, heading, button, or input element) to the end of the page's content list. arXiv

`extend()` : Adds multiple items from an iterable or another list to the end of the content list.

`insert()` : Adds a component or element at a specific index within the page layout.

List Concatenation (`+` or `+=`): Merges additional components or sub-lists into the page's content list.

3. Website-Level Content Additions

For global elements that appear outside the individual route's content area (such as custom scripts, headers, or stylesheets), Drafter provides top-level configuration functions:

`add_website_header()` : Injects raw HTML elements directly into the document header. PyPI

`add_website_css()` : Injects raw CSS rules or styling into the page.

`add_website_css_file()` : Links an external stylesheet or CSS file. PyPI

`add_website_js()` : Adds executable JavaScript code to the page runtime. PyPI

`add_website_js_file()` : Links external JavaScript source files. PyPI

`add_website_file()` : Declares auxiliary asset files needed by the application.

how can i make each appended item appear under the others, not next to it in python

In Drafter, whether elements display side-by-side or stacked vertically depends on whether the component renders as an **inline element** (like plain text or a `Text` component) or a **block element** (like `Paragraph`, `Header`, or `Box`).

Here are the ways to make appended items stack vertically:

1. Wrap Each Item in a `Paragraph`

By default, plain text or basic strings sit inline next to each other. Wrapping each element inside Drafter's `Paragraph` component forces it to render as a block element on its own dedicated line.

2. Group Each Item in a `Box`

The `Box` container component renders as a block container (`<div>`). Placing an item or a group of items inside its own `Box` causes each one to start on a new line directly below the previous one.

3. Use `Header` Components

If the items represent titles, steps, or labels, Drafter's header components (`Header` , `Subheader` , or numbered header equivalents) naturally behave as block elements and automatically stack.

4. Separate with Line Breaks (`HorizontalRule` or `br`)

If you prefer not to change the components themselves:

- Append a line break or Drafter's `HorizontalRule` between items to explicitly push the next item to the following line.
- Use a newline character (`\n`) if you are rendering preformatted or formatted text blocks.

5. Apply Block CSS Display

If using custom styling via Drafter's CSS support, applying the `display: block` style to your component class or inline style properties will force any naturally inline element to take

what are the different ways text can be displayed on screen, like bold and italics

Text on screen can be styled across several distinct dimensions: weight, posture, decoration, casing, spacing, and typographic effects.

1. Font Style & Weight

- **Bold / Heavy / Black**: Thickened stroke weights used for emphasis, titles, and visual hierarchy.
- **Italics / Oblique**: Slanted text often used for emphasis, book titles, quotes, or terminology.
- **Light / Thin**: Reduced stroke weight, often used for subtle secondary labels or modern minimalist headlines.

2. Text Decoration

Underline: A line running underneath the text, traditionally indicating hyperlinks or legal emphasis.

Strikethrough / Line-Through: A horizontal line directly through the middle of the characters, commonly used to indicate deleted, completed, or deprecated items.

Overline: A horizontal line drawn above the text, occasionally used in technical or mathematical contexts.

3. Position & Vertical Alignment

Superscript (X^2): Scaled-down text set slightly above the normal text baseline, standard for footnotes, exponents, and ordinal indicators (1st).

Subscript (H_2O): Scaled-down text set slightly below the baseline, standard for chemical formulas and mathematical indices.

4. Capitalization & Case Transformation

All Caps / Uppercase: Every letter capitalized, commonly used for alerts, acronyms, or short headings.

Small Caps: Capital glyphs sized to match the height of lowercase letters (x-height), standard in formal book typography and abbreviations.

Title Case / Capitalized: Capitalizing the first letter of each significant word.

Lowercase: Forcing all characters into small letters.

5. Font Family & Classification

Serif: Fonts with decorative strokes ("feet") at the ends of letterforms (e.g., Times New Roman, Georgia), common in formal reading.

Sans-Serif: Clean fonts without strokes (e.g., Arial, Inter, Helvetica), the web and UI standard for legibility.

Monospace: Fonts where every character has an identical width (e.g., Courier, Consolas), standard for code, terminal output, and aligned data tables.

Display / Script: Decorative or handwritten typefaces reserved for brand identity, logos, or posters.

6. Visual Effects & Treatment

Highlighted / Inverted: Adding a colored background behind text (similar to a physical highlighter marker) to call out search hits or key points.

Color & Opacity: Tinting text with custom colors or lowering opacity/contrast for secondary or disabled states.

Drop Shadow / Glow: Adding blurred silhouettes behind letterforms to enhance readability over complex image backgrounds.

Letter Spacing (Tracking): Expanding or tightening the horizontal gaps between characters.

How would I format a bold text

In Drafter, text formatting is handled primarily through built-in helper functions that style text before adding it to a page or component:

1. The `bold()` Function

Drafter provides a dedicated text-styling function called `bold()`. You pass a string (or a string converted from another data type) into `bold()`, and it returns a bolded text object. This formatted result can be passed directly into a `Page` or into layout components like `Div` or `Box`. Dodona + 1

2. Composing with Other Text Modifiers

Because Drafter's styling helpers are standard functions, you can nest or compose them together. For example, you can wrap text in `bold()` alongside other styling functions such as `italic()` or `change_color()` to apply multiple styles simultaneously. Dodona

3. Placing Bold Text Inline

If you want only a specific word or phrase bolded within a longer sentence, wrap the specific piece of text inside `bold()` and place it alongside regular strings inside a

container component like `Div` . This keeps the surrounding sentence structure intact while emphasizing the targeted phrase.

4. Non-String Data

If you want to bold dynamic application data (such as a numeric score or a state variable), convert the value to a string first using standard type conversion (e.g., `str()`) before passing it into `bold()` , as the styling functions expect string input. Dodona

If you are exploring graphical interface design more broadly in Python, the Tkinter Text Widget Styling Tutorial demonstrates how text formatting tags are configured and applied in

Can you make one unit test per function in the following program using assert_equal or assert statements:

from dataclasses import dataclass

Here are the unit tests for each function in your program using Drafter's `assert_equal` function (you can also use standard Python `assert` interchangeably).

Each test sets up an isolated `State` instance, executes the target function, and asserts the expected return values, content elements, or updated state attributes.

Python

```python
from drafter import assert_equal


def test_make_navbar():
    state = State([], "Alice", "All", "Cache cleared", False)
    nav = make_navbar(state)

    expected = [
        Button("Dashboard", index),
```

```
            Button("Host Roster", host_roster),
            Button("Provision New Host", provision_host_page),
            Button("Vulnerability Audit", vulnerability_audit),
            Button("Maintenance", maintenance_page),
            "\n",
            bold("[System Log: Cache cleared]"),
            "\n\n",
        ]
        assert_equal(nav, expected)


    def test_index():
        host = Host("web-01", "10.0.0.1", "Linux", 2, False, "Web server")
        state = State([host], "Bob", "All", "", False)
        page = index(state)

        expected_content = [
            Button("Dashboard", index),
            Button("Host Roster", host_roster),
            Button("Provision New Host", provision_host_page),
            Button("Vulnerability Audit", vulnerability_audit),
            Button("Maintenance", maintenance_page),
            "\n",
            bold("Network Inventory Database"),
            "\n",
            "System Administrator: ",
            bold("Bob"),
            "\nMaintenance Window: INACTIVE (Normal Operations)\n",
            "Total hosts currently monitored: 1",
        ]
        assert_equal(page, Page(state, expected_content))


    def test_maintenance_page():
        state = State([], "Admin", "All", "", True)
        page = maintenance_page(state)

        expected_content = [
            Button("Dashboard", index),
            Button("Host Roster", host_roster),
            Button("Provision New Host", provision_host_page),
            Button("Vulnerability Audit", vulnerability_audit),
            Button("Maintenance", maintenance_page),
            "\n",
            bold("Maintenance Settings"),
            "\n",
```

```
            "Current Maintenance Status: ",
            bold("ENABLED"),
            "\n",
            Button("Toggle Maintenance Mode", toggle_maintenance),
            "\n",
            "-------------------------------------------------\n",
            bold("Administrator Settings\n"),
            "System Administrator: Admin\n",
            "Update Admin Operator Name: ",
            TextBox("new_admin", "Admin"),
            Button("Update Name", update_admin),
        ]
        assert_equal(page, Page(state, expected_content))


    def test_update_admin():
        state = State([], "OldAdmin", "All", "", True)
        page = update_admin(state, "NewAdmin")

        assert_equal(state.admin_name, "NewAdmin")
        assert_equal(state.status_log, "Admin operator changed to: NewAdmin")
        assert_equal(page, maintenance_page(state))


    def test_toggle_maintenance():
        state = State([], "Admin", "All", "", False)
        page = toggle_maintenance(state)

        assert_equal(state.maintenance_mode, True)
        assert_equal(state.status_log, "Maintenance Mode is now set to: True")
        assert_equal(page, maintenance_page(state))


    def test_host_roster():
        host = Host("gw-01", "192.168.1.1", "Cisco IOS", 1, True, "Core switch")
        state = State([host], "Admin", "Cisco IOS", "", False)
        page = host_roster(state)

        expected_content = [
            Button("Dashboard", index),
            Button("Host Roster", host_roster),
            Button("Provision New Host", provision_host_page),
            Button("Vulnerability Audit", vulnerability_audit),
            Button("Maintenance", maintenance_page),
            "\n",
            bold("Network Host Roster"),
```

```
            "\n",
            "Filter by Operating System: ",
            SelectBox("os_filter", OS_FILTER_OPTIONS, "Cisco IOS"),
            Button("Apply OS Filter", apply_os_filter),
            "\n",
            "gw-01 (192.168.1.1) | OS: Cisco IOS",
            " | Ports: 1 | ",
            bold("[PATCH REQUIRED]"),
            "\n",
            italic("Notes: Core switch"),
            "\n",
            "-------------------------------------------------\n",
            bold("Toggle Patch Status for a Host:\n"),
            "Enter Exact Hostname: ",
            TextBox("target_hostname", ""),
            Button("Toggle Patch Status", toggle_patch_status),
            "\n",
        ]
        assert_equal(page, Page(state, expected_content))


    def test_apply_os_filter():
        state = State([], "Admin", "All", "", False)
        page = apply_os_filter(state, "Linux")

        assert_equal(state.selected_os_filter, "Linux")
        assert_equal(state.status_log, "Host filter set to: Linux")
        assert_equal(page, host_roster(state))


    def test_toggle_patch_status():
        host = Host("db-01", "10.0.0.5", "Linux", 1, False, "Main DB")
        state = State([host], "Admin", "All", "", False)
        page = toggle_patch_status(state, "db-01")

        assert_equal(host.needs_patching, True)
        assert_equal(state.status_log, "Toggled patch status for: db-01")
        assert_equal(page, host_roster(state))


    def test_provision_host_page():
        state = State([], "Admin", "All", "", False)
        page = provision_host_page(state)

        expected_content = [
            Button("Dashboard", index),
```

```
            Button("Host Roster", host_roster),
            Button("Provision New Host", provision_host_page),
            Button("Vulnerability Audit", vulnerability_audit),
            Button("Maintenance", maintenance_page),
            "\n",
            bold("Provision New Network Asset"),
            "\n",
            "Hostname:\n",
            TextBox("hostname", "srv-new-01"),
            "\nIP Address:\n",
            TextBox("ip_address", "192.168.1.50"),
            "\nOperating System:\n",
            SelectBox("os_type", OS_OPTIONS, OS_OPTIONS[0]),
            "\nOpen Port Count (Estimated):\n",
            TextBox("port_count_str", "2"),
            "\nNeeds Patch: ",
            CheckBox("needs_patching", False),
            "\nRunning Services & Notes:\n",
            TextArea("notes", "SSH (22), Web Server (80)"),
            "\n",
            Button("Provision Host", save_new_host),
        ]
        assert_equal(page, Page(state, expected_content))


    def test_save_new_host():
        state = State([], "Admin", "All", "", False)
        page = save_new_host(
            state,
            "backup-01",
            "192.168.1.99",
            "Linux",
            "3",
            False,
            "Storage node",
        )

        expected_host = Host("backup-01", "192.168.1.99", "Linux", 3, False, "Storage
        assert_equal(state.hosts, [expected_host])
        assert_equal(state.status_log, "Successfully provisioned host: backup-01")
        assert_equal(page, host_roster(state))


    def test_vulnerability_audit():
        host1 = Host("app-01", "10.0.0.10", "Linux", 5, True, "Vulnerable service")
        host2 = Host("app-02", "10.0.0.11", "Linux", 2, False, "Safe node")
```

```
        state = State([host1, host2], "Admin", "All", "", False)
        page = vulnerability_audit(state)

        expected_content = [
            Button("Dashboard", index),
            Button("Host Roster", host_roster),
            Button("Provision New Host", provision_host_page),
            Button("Vulnerability Audit", vulnerability_audit),
            Button("Maintenance", maintenance_page),
            "\n",
            bold("Network Vulnerability Audit"),
            "\n",
            "- Monitored Assets: 2\n",
            "- Total Exposed Open Ports: 7\n",
            "- Unpatched Assets: 1\n",
            "- High-Risk Devices (Unpatched and >3 Open Ports): ",
            bold("1"),
            "\n",
        ]
```